

**LAPORAN PROYEK UAS
ALGORITMA PEMROGRAMAN
“SIMULASI ATM SEDERHANA”**



Kelompok “MESIN ATM” :

- 1. Adi Jaya Wibawa (252011114)**
- 2. Jefry Adriano Alkautsar (252011104)**

**PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI DAN DESAIN
INSTITUT TEKNOLOGI DAN BISNIS ASIA MALANG
2026**

1. ANALISIS MASALAH

a. Latar Belakang

Dalam era digital saat ini, teknologi finansial memang menjadi bagian tak terpisahkan dari kehidupan sehari-hari, salah satunya adalah mesin Anjungan Tunai Mandiri (ATM). ATM bekerja sebagai sistem yang melayani transaksi perbankan secara otomatis dengan logika yang ketat dan aman. Meskipun terlihat sederhana bagi pengguna, di balik layar terdapat struktur algoritma yang kompleks untuk menangani validasi PIN, pengecekan saldo, hingga aritmatika penarikan dan penyetoran uang.

Bagi seorang *software developer*, memahami alur kerja sistem seperti ATM merupakan studi kasus yang menarik untuk melatih logika pemrograman. Simulasi ATM sederhana menawarkan representasi nyata dari operasi CRUD sendiri.

Oleh karena itu, proyek simulasi ATM ini dibuat menggunakan bahasa pemrograman Python. Python dipilih karena sintaksisnya yang ringkas dan mudah dibaca, memungkinkan fokus utama tertuju pada pengembangan algoritma dan struktur data yang efisien dalam menangani skenario transaksi perbankan dasar.

b. Tujuan

- **Memahami Logika Sistem Transaksi:** Menganalisis dan menerapkan algoritma dasar yang digunakan dalam sistem ATM, seperti validasi pengguna (login), logika pengurangan saldo (penarikan), penambahan saldo (setor tunai) dan transaksi antar user.
- **Implementasi Struktur Kontrol Python:** Menerapkan konsep dasar pemrograman Python, khususnya penggunaan percabangan (conditional statement if/else) untuk pengambilan keputusan dan perulangan (looping) untuk menjaga sesi transaksi tetap berjalan.
- **Manajemen Data Sederhana:** Melatih kemampuan dalam mengelola variabel dan struktur data untuk menyimpan informasi pengguna dan saldo menggunakan JSON untuk sistem manajemen data sederhana.
- **Simulasi Penyelesaian Masalah (Problem Solving):** Melatih kemampuan berpikir komputasional dalam menangani skenario kesalahan (error handling), misalnya bagaimana sistem merespons jika saldo tidak mencukupi atau PIN salah.

2. FLOWCHART

ALGORITMA SIMULASI ATM SEDERHANA

Tahap Inisialisasi

- inisialisasi variabel :
 - file_json = 'user.json' sebagai file database pengguna

ALGORITMA FUNGSI load_data()

Mulai

- Periksa apakah path/lokasi file (file_json) tersedia di sistem operasi :
 - JIKA TIDAK ADA: Hentikan proses segera dan kembalikan nilai default berupa list kosong [] (Early Return).
 - JIKA ADA: Buka file tersebut | Baca file.json dan inisialisasi sebagai objek file. | ubah isi teks file menjadi tipe data List/Dict Python. | Kirim hasil data tersebut keluar dari fungsi.
- JIKA GAGAL (karena file mendadak hilang/FileNotFoundError atau isinya rusak/JSONDecodeError):
 - Tangkap error tersebut agar program tidak crash.
 - kembalikan dengan list kosong []

Selesai

ALGORITMA FUNGSI save_data(all_data)

Mulai

- Buka file user.json
- Simpan seluruh data user ke dalam file dalam format JSON
- Jika terjadi kesalahan:
 - Tampilkan pesan gagal menyimpan data

Selesai

ALGORITMA FUNGSI convert_uang(angka)

Mulai

- Ubah angka menjadi format rupiah:
- Kembalikan hasil format rupiah

Selesai

ALGORITMA LOGIN USER (login(database_user))

Mulai

- Inisialisasi:
 - chance = 1
 - max_chance = 3
- Masuk ke perulangan selama chance <= max_chance:
 - Tampilkan informasi login
 - Inputkan Akun Bank (3 digit), dan Pin ATM (6 digit)
 - Jika akun bank bukan 3 digit:
Tampilkan pesan kesalahan, tambahkan chance -> Kembali ke login

- Jika PIN bukan 6 digit:
Tampilkan pesan kesalahan, tambahkan chance -> Kembali ke login
- Cek database:
 - Untuk setiap user di database:
 - Jika akun_bank dan pin cocok:
Tampilkan pesan login berhasil, Kembalikan data user, Keluar dari fungsi
 - Jika tidak ditemukan akun yang cocok:
Tampilkan pesan "akun atau pin salah", Tambahkan chance salah
 - Jika input bukan angka:
Tampilkan pesan input tidak valid, Tambah chance salah
 - Jika chance $\geq$ max_chance:
Tampilkan pesan percobaan habis, Keluar dari fungsi (kembalikan None)

ALGORITMA Fungsi transfer(current_users, database_users)

- Mulai Fungsi
- Input Rekening: Minta pengguna memasukkan nomor rekening tujuan.
- Jika input bukan angka,
tampilkan pesan error dan ulangi langkah 1.
- Pencarian Rekening: Cari nomor rekening di dalam all_data.
- Jika tidak ditemukan:
Tampilkan pesan "Rekening tidak ditemukan" dan kembali ke langkah
- Jika ditemukan:
Cek apakah itu rekening sendiri. Jika ya, tampilkan error dan kembali ke langkah
- Jika bukan rekening sendiri, lanjut ke tahap konfirmasi.
Konfirmasi Pemilik Rekening: Tampilkan nama dan nomor rekening penerima.
- Minta pengguna memilih (1. BENAR / 2. SALAH).
 - Jika SALAH (2): Batalkan transfer dan kembali ke langkah memilih rekenin
 - Input Nominal: Jika konfirmasi benar, minta masukkan jumlah nominal transfer.
- Jika input bukan angka
ulangi permintaan nominal.
- Konfirmasi Nominal: Tampilkan jumlah yang akan ditransfer.
- Minta pengguna memilih (1. BENAR / 2. SALAH).
 - Jika SALAH (2)
Batalkan konfirmasi nominal dan kembali ke langkah sebelumnya
- Eksekusi Transfer: * Cek apakah saldo user cukup.
- Jika Saldo Cukup:
- Kurangi saldo pengirim (user).
- Tambah saldo penerima (penerima_ditemukan).
- Panggil save_data(all_data) untuk memperbarui database.
- Tampilkan pesan "Transfer Berhasil".
- Jika Saldo Kurang: Tampilkan pesan "Saldo tidak cukup".
- Selesai: Keluar dari fungsi transfer (return).

ALGORITMA FUNGSI check_balance(user)

Mulai

- Ambil saldo dari user (user dari database json)
- Ubah ke format rupiah
- Tampilkan saldo
- Jeda 1 detik

Kembali ke menu

ALGORITMA FUNGSI deposit(user, all_data, amount)

Mulai

- Jika amount > 0:
Tambahkan amount ke saldo user, Simpan data ke JSON menggunakan fungsi save_data(all_data)
- Tampilkan setor tunai dan saldo terbaru
- Jika tidak:
Tampilkan pesan deposit gagal
- Jeda 1 detik
- Kembali ke menu

Selesai

ALGORITMA FUNGSI withdraw(user, all_data, amount)

Mulai

- Jika $0 < \text{amount} \leq \text{user}$
Kurangi Saldo user, Simpan data ke Json menggunakan fungsi save_data(all_data)
- Tampilkan saldo terbaru
- Jika tidak:
Tampilkan pesan penarikan gagal,
- Jeda 1 detik
- Kembali ke menu

Selesai

ALGORITMA MENU UTAMA (main_menu())

- Inisialisasi:
 - current_user
 - database_users
- Tampilkan judul program
- Masuk ke perulangan tak hingga (while True)

- Tampilkan menu:
 1. Cek Saldo
 2. Setor Tunai
 3. Tarik Tunai
 4. Keluar
- Input pilihan user → choice
 - Jika choice antara 0–4:
 - Jika choice == 0:
Tampilkan pesan keluar, Hentikan program
 - Jika choice == 1:
Panggil fungsi check_balance()
 - Jika choice == 2:
Input jumlah setor, Panggil deposit()
 - Jika choice == 3:
Input jumlah tarik, Panggil withdraw()
 - Jika choice == 4:
Panggil fungsi transfer()
 - Jika pilihan tidak sesuai / tidak berupa integer:
Tambahkan counter_spam, tampilkan peringatan spam
 - Jika counter_spam >= max_spam:
Tampilkan pesan blokir, keluar dari program
 - Jika input bukan angka:
Tambahkan spam counter, tampilkan pesan kesalahan

ALGORITMA PROGRAM UTAMA (__main__)

Mulai Program

- Panggil load_data() → simpan ke variabel database_users
- Panggil login(database_users) → simpan ke variabel current_user
- Jika current_user tidak kosong:
 - Panggil main_menu(current_user, database_users)
- Jika gagal login:
 - Program berhenti

PROGRAM BERAKHIR JIKA:

- User memilih menu Keluar
- Login gagal 3 kali
- User melakukan spam input
- Terjadi error fatal


```

except (json.JSONDecodeError, FileNotFoundError): #jika error maka diberi list
kosong
    return []

```

```

def save_data(all_data): #ini diartikan sebagai untuk menyimpan datanya
    try:
        with open(file_json, 'w') as file: #akses filenya dengan write mode
            json.dump(all_data, file, indent=4) #lalu tempel ke seluruh datanya ke file
            dengan indent 4, indent 4 hanya biar baris key dan valuenya dipisah saja
        except Exception as e: #jika error maka tampilkan
            print (f"Gagal menyimpan data: {e}")

```

```

# =====

```

```

# 2. BAGIAN BANTU BANTU

```

```

# =====

```

```

def convert_uang(angka): # ini fungsi untuk convert uang
    hasil_awal = f"Rp {angka:,.0f}" #angka sebagai parameter yang masuk akan
    diubah menjadi Rp, dengan tanpa desimal
    hasil_akhir = hasil_awal.replace(',', '.') #mengganti (,) karna standar US ke (.)
    standar Indo

```

```

    return hasil_akhir #lalu direturn nilainya sebagai hasil akhir

```

```

# =====

```

```

# 3. BAGIAN AUTH (Login)

```

```

# =====

```

```

def login(database_user): #fungsi ini sebagai login user
    chance = 1 # dengan chance atau kesempatan valuenya set sebagai 1
    max_chance = 3 #max chance untuk membatasinya kita set sebagai 3

```

```

    while (chance <= max_chance): # intinya disini jika chance nya melebihi 3 maka
    programnya akan selesai

```

```

        try:
            print(f"\n--- Percobaan Login ke-{chance} dari {max_chance} ---")
            print(f""""
"Selamat Pagi", Selamat datang di Simulasi ATM SDERHANAAAAA!!!!

Silahkan Masukkan Akun Bank Beserta Pinnya
""")

```



```
account_input = input(f"Masukkan Akun Bank kamu (3 digit) : ") #disini kita harus menginputkan 3 agnak
```

```
account_angka = int(account_input) #dengan batasan kita harus menginputkan integer
```

```
if len(account_input) != 3: #ini pembatasan angkanya harus 3  
    print (f"Format Salah Akun Bank harus 3 digit") #ini adalah format salah jika lebih dari 3 atau kurang dari 3
```

```
    time.sleep(1) #jeda 1 detik
```

```
    chance += 1 # chance ditambah 1
```

```
    continue # lalu kembali memasukkan akun bank
```

```
password = input(f"Masukkan PIN ATM kamu (6 digit) : ") #ini kita harus memasukkan Pin
```

```
password_angka = int(password) #dibatasi dengan integer
```

```
if len(password) != 6: #ini pembatasan agar di set cmn 6 saja
```

```
    print (f"Format Salah Akun Bank harus 3 digit") #ini teks ketika salah
```

```
    time.sleep(1) #lalu kita jeda 1 detik
```

```
    chance += 1 #lalu ditambahkan 1 untuk kesempatannya
```

```
    continue #ulangi lagi untuk mmeasukkan akun bank, bukan pin lagi
```

```
for user in database_user: #ini untuk mencari apakah user akun bank input, dan pin input ada dalam database
```

```
    if user['akun_bank'] == account_angka and user['pin'] == password_angka: #ini decision jika ada
```

```
        print (f"\nLogin Berhasil Selamat datang {user['nama']}") #maka login akan berhasil
```

```
        time.sleep(1) # tetap di jeda untuk 1 detiknya
```

```
        return user #lalu kita kembalikan sebagai variabel user
```

```
    print("Akun atau Pin yang Anda masukkan salah, coba lagi") # jika salah pin dan akun yang dimasukan maka muncul teks ini
```

```
    chance += 1 # chance ditambah 1
```

```
    time.sleep(1) # jeda 1 detik
```

```
except ValueError: #jika mendapatkan inputan error maka
```

```
    print("✗ Input tidak valid. Kamu harus memasukkan angka.") # akan muncul pesan ini karna inputan dirasa tidak valid
```

```
    chance += 1 # chance ditambah 1
```

```
    time.sleep(1) #lalu jeda 1 detik
```

```

        if chance > max_chance: #ini decision untuk pembatasan kesempatan login,
jika melebihi 3 maka
            print (f"Anda telah melakukan percobaan {max_chance} kali, coba ulangi
beberapa saat lagi") # akan muncul pesan ini
            break #program selesai
        return None #dan loopingan ini akan dikembalikan dengan none atau kosongan
saja

```

```

# =====

```

```

# 4. BAGIAN TRANSAKSI

```

```

# =====

```

```

def check_balance(user): #ini fungsi check balance
    convert_balance = convert_uang(user['balance']) #ini convert uang agar ke
nominal rupiah
    print(f"Saldo Anda saat ini adalah: {convert_balance}") # ini agar saldo nya tampil
dan menjadi rupiah
    time.sleep(1) # jeda 1 detik

```

```

def deposit(user, all_data, amount): # fungsinya ini untuk deposit atau
menambahkan saldo

```

```

    if amount > 0: # jika amount lebih besar dari nol at least 1 saja
        user['balance'] += amount #code ini akan dieksekusi jadi ditambahkan ke json
        save_data(all_data) #fungsi save data tetap ditambahkan agar di json
terupdate

```

```

        convert_amount = convert_uang(amount) #ini convert untuk mata uang
        convert_balance = convert_uang(user['balance']) #ini agar saldo user adar
rupiahnya

```

```

        print(f"Setor Tunai: {convert_amount}. Update Saldo: {convert_balance}")
#ketika ditampilkan jadi lebih bagus

```

```

    else:
        print("Deposit Gagal. Jumlah harus lebih dari 0.") #jika gagal maka akan
mengesekusi pesan ini
        time.sleep(1) #jeda 1 detik

```

```

def withdraw(user, all_data, amount): # fungsi ini untuk melakukan penarikan saldo
dari user

```

```

    if 0 < amount <= user['balance']: #decision ini mengamankan agar ketika saldo
user ketika kosong maka tidak bisa ditarik

```

```

        user['balance'] -= amount # jika if berhasil terksekusi akan lanjut pengurangan
        saldo user
        save_data(all_data) #lalu data akan disimpan di json menggunakan fungsi ini
        convert_amount = convert_uang(amount) # ini untuk mengubah amount ke
        rupiah biar menarik
        convert_balance = convert_uang(user['balance']) #ini convert rupiah untuk
        saldo user
        print(f"Tarik Tunai: {convert_amount}. Update Saldo: {convert_balance}") #lalu
        ini pesan yang akan ditampilkan
    else:
        print("Penarikan Gagal. Periksa jumlah yang Anda masukkan dan saldo Anda.")
        time.sleep(1) #jeda 1 detik

```

```

def transfer (user, all_data): #ini fungsi transfer
    while True: #ini digunakan agar tidak keluar fungsi ini saja
        akun_orang = input("Masukkan Nomor Rekening Tujuan : ") #meginputkan
        rekening tujuan dulu
        try:
            rekening_tujuan = int(akun_orang) #ini diset agar memasukkan integer
        except ValueError: #ini bagian menangkap jika ada code error
            print("Inputan harus Berupan angka") #ini pesan yang akan ditampilkan
            continue # ini agar user mengulang untuk memasukkan akun bank jika salah

```

```

        penerima_ditemukan = None #Variabel untuk menyimpan data penerima
        sebagai nilai kosongan
        for data_orang in all_data: #mencari data orang yang dituju di database
            if data_orang['akun_bank'] == rekening_tujuan: #mencari jika data orang di
            akun bank (Database) sama dengan yang diinputkan
                penerima_ditemukan = data_orang #jika ditemukan set dulu sebagai
                data orang
                break # lalu lanjut ke code selanjutnya

```

```

        if penerima_ditemukan: #ini decision jika penerima ditemukan dari variabel
        none tadi sebagai pengaman saja
            if penerima_ditemukan['akun_bank'] == user['akun_bank']: # ini dicek akun
            bank sama user akun bank itu sama atau tidak
                print("Gagal: Anda tidak bisa transfer ke rekening sendiri.") # jika sama
                maka akan muncul pesan error ini agar kita mengetahui kalo user tidak transaksi
                antar user
                time.sleep(1) #jeda 1 detik

```

```

        continue #lalu ini mengulangi memasukkan akun bank lagi yang ingin
dituju
        nama_penerima = penerima_ditemukan['nama'] #ini set nama penerima nya
dari database
        rekening_penerima = penerima_ditemukan['akun_bank'] # ini set rekening
penerima dari database
        balance_penyetor = user['balance'] #ini set balance dari user di database,
tujuannya ini agar mudah aja untuk diketik
        print(f"""
KONFIRMASI TRANSFER
Ke: {nama_penerima}
Rek: {rekening_penerima}

```

1. BENAR

2. SALAH

```

""") #menampilkan pesan konfirmasi

```

```

        while True: #ini digunakan agar inputan tadi tidak ada yang missing sama
sekali jadi cukup 1 atau 2 saja

```

```

        try:
            konfirmasi_akun = int(input("Masukkan Pilihan (1/2): ")) #ini kita input 1 /
2 saja

```

```

            if konfirmasi_akun == 1 : #jika memilih 1 akan mengesekusi baris
bawahnya

```

```

                while True: #dengan loop tak hingga
                    nominal = input(f"""
Saldo kamu sekarang adalah : {balance_penyetor}
Masukkan nominal transfer untuk {nama_penerima}: """) # menampilkan pesan
singkat saja

```

```

                    try :
                        nominal = int(nominal) #dan ini membatasi agar jadinya hanya
integer saja

```

```

                    except: #ini ketika ada salah input mereka harus input ulang
                        print("Nominal tidak valid (harus angka). Silakan input ulang.")
                        continue # Kembali ke input nominal (Loop 2)
                        nominal_convert = convert_uang(nominal) # ini convert biar rapi ke
rupiah

```

```

                    print(f"""
KONFIRMASI TRANSFER
Ke: {nama_penerima}
Jumlah Transfer : {nominal_convert}

```

1. BENAR

2. SALAH

""") # ini pesan untuk konfirmasi

while True : #ini masuk ke loop tak hingga lagi

try: # try ini menangkap jika ada salah

nominal_ask= int(input("Masukkan pilihan (1/2): ")) # ini

membatasi jika input hanya 1 / 2

if nominal_ask == 1: # jika nominal 1

--- EKSEKUSI TRANSFER DISINI ---

print("Transfer SEDANG DIPROSES...")

if user['balance'] >= nominal: #ini mengecek apakah saldo user lebih besar sama dengan dari nominalnya

user['balance'] -= nominal #maka code ini akan mengsekusi ini dan mengurangi saldo user

penerima_ditemukan['balance'] += nominal

convert_balance = convert_uang(user['balance'])

save_data(all_data)

print(f"Transfer Berhasil! Sisa saldo anda: {convert_balance}")

return

else: # jika gagal maka akan mengesekusi tampilan ini artinya user tidak memiliki saldo

convert_balance = convert_uang(user['balance'])

print(f"Saldo tidak cukup Sisa saldo anda: {convert_balance}")

...

print("Transfer BERHASIL.")

return # Selesai, keluar dari fungsi transfer

elif nominal_ask == 2: #jika memilih 2 maka menampilkan baris pesan ini

print("Konfirmasi dibatalkan. Silakan masukkan nominal yang benar.")

break #ini akan membuat keluar ke fungsi konfirmasi dan user diminta memasukkan nominal kembali dengan benar

else: #jika inputan 3 or etc maka akan membuat pesan ini muncul dan user hanya boleh memilih 1 atau 2

print("Pilihan salah. Hanya 1 atau 2.")

continue

except ValueError: #ini akan menangkap segala error agar code tetap berjalan

```

        print("Error: Masukkan angka saja!")
    elif konfirmasi_akun == 2: #ini bagian konfirmasi yang salah jika, salah
    transfer akan dibatalkan
        print(f"Permintaan transfer dibatalkan, Silahkan coba lagi")
        break #lalu keluar dari fungsi konfirmasi dan user diminta
    memasukkan akun bank dengan benar
    else: #jika ada inputan salah maka akan memunculkan pesan ini
        "Input salah. Masukkan angka 1 atau 2." #inputan hanya berupa
    angka 1 atau 2 saja
    except ValueError: #ini menangkap jika ada salah satu value yang error
    maka akan memunculkan pesan ini
        print("Input salah. Masukkan angka 1 atau 2.")
    else: #ini else jika rekening yang dimasukkan tidak ketemu atau penginput
    memasukkan salah akun bank
        print(f"Nomor Rekening {rekening_tujuan} tidak ditemukan, silahkan coba
    lagi")
        continue

def main_menu(current_users, database_users): #ini menu utama user
    print(f"""
    =====|
    Selamat Datang di Simulasi ATM Sederhana|
    =====|
    """)
    while True:
        print("\nMenu:")
        print("""
    0. Keluar
    1. Cek Saldo
    2. Setor Tunai
    3. Tarik Tunai
    4. Transfer
    """)
        try:
            choice = input("Pilih opsi (0-4): ") # user hanya boleh menginputkan 0 - 4 saja
            choice_convert = int(choice) # ini agar integer

            # Mendeklarasikan agar opsi 1 dan 4 mereset counter spam menjadi 0, agar
    tidak dihitung kemudian menjadi 1 lagi.
            if 0 <= choice_convert <= 4: # ini decision jika choice hanya boleh 0 - 4
                if choice_convert == 0: #jika 0 maka program akan berhenti

```

```
print("Terima kasih telah menggunakan Layanan ATM Sederhana, Adios  
Mabroo!")
```

```
break #ini membuat program berhenti  
elif choice_convert == 1: #jika pilih angka "1", maka akan masuk ke fungsi  
check_balance
```

```
    check_balance(current_users)  
    elif choice_convert == 2: #jika pilih angka "2", maka bisa memasukkan  
nilai ke variabel amount, lalu nilai tersebut dibuat parameter, ketika fungsi deposit  
dijalankan.
```

```
        amount = int(input("Masukkan jumlah yang akan disetorkan dalam  
bentuk rupiah: "))
```

```
        deposit(current_users, database_users, amount)  
    elif choice_convert == 3: #jika pilih angka 3 maka user memasukkan nilai  
ke variabel amount lalu memanggil fungsi withdeaw
```

```
        amount = int(input("Masukkan jumlah yang akan ditarik dalam bentuk  
rupiah: "))
```

```
        withdraw(current_users, database_users, amount)  
    elif choice_convert == 4: # jika user memasukkan angka 4 maka akan  
masuk ke fungsi tranfer
```

```
        transfer(current_users, database_users)  
    else : # jika tidak ada dalam pilihan ini maka user akan diminta memasukkan  
pilihan yang valid
```

```
        print("❌ Pilihan tidak valid. Silahkan coba lagi.") #  
        time.sleep (1) # jeda 1 detik
```

```
except ValueError : # ini menampung semua pilihan yang tak valid atau jika ada  
string maka akan dieksekusi pesan ini
```

```
    print("❌ Input tidak valid. Kamu harus memasukkan angka 0 - 4.")  
    time.sleep (1) # jeda 1 detik
```

```
except Exception as e: #jika ada kesalahan tak terduga sebagai developer akan  
memunculkan pesan dilayar, tapi tidak baik ketika release untuk umum
```

```
    print(f"⚠️ Terjadi kesalahan tak terduga: {e}")  
    break # ini yang membuat program berhenti
```

```
# --- Menjalankan Program ---
```

```
if __name__ == "__main__": #ini bagian menjalankan program dengan main di  
utamanya
```

```
    database_users = load_data() #ini set fungsi load data tadi sebagai  
database_users
```

```
current_user = login(database_users) #akan menjadi current user ketika akan login
```

```
if current_user is not None: #jika ketika login dan bukan none yang di return maka akan masuk ke main menu
```

```
    main_menu(current_user, database_users) #ini main menu user dengan data yang di load tadi dan user sekarang
```